LIST

- If we want to represent a group of individual objects as a single entity where insertion order preserved and duplicates are allowed, then we should go for List.
- insertion order preserved.
- duplicate objects are allowed.
- heterogeneous objects are allowed.
- List is dynamic because based on our requirement we can increase the size and decrease the size.
- In List the elements will be placed within square brackets and with comma separator.
- Python supports both positive and negative indexes. +ve index means from left to right where as negative index means right to left.
- List objects are mutable. i.e we can change the content.

```
my_list = [10, "A", "B", 20,  30, 10]
```

-6	-5	-4	-3	-2	-1
10	A	B	20	30	10
0	1	2	3	4	5

Creation of List Objects:

(1) We can create empty list object as follows.

```
list=[]
print(list)
print(type(list))
```

(2) If we know elements already then we can create list as follows

list = [10, 20, 30, 40]

(3) With list() Function:

```
l=list(range(0,10,2))
print(l)
print(type(l))
```

(4) With split() Function:

```
s="Learning Python is very very easy !!!"
l=s.split()
print(l)
print(type(l))
print(len(l))
```

(5) Sometimes we can take list inside another list, such type of lists are called nested lists.

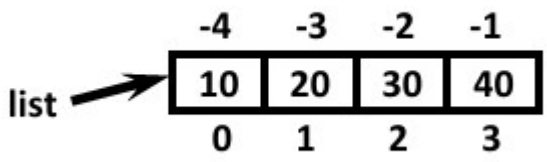
[10, 20, [30, 40]]

Accessing Elements of List:

We can access elements of the list either by using index or by using slice operator

(1) By using Index:

- **By using Index:**
- **List follows zero based index. ie index of first element is zero.**
- **List supports both +ve and -ve indexes.**
- **+ve index meant for Left to Right**
- **-ve index meant for Right to Left**
- **list = [10, 20, 30, 40]**



print(list[0]) → 10

print(list[-1]) → 40

print(list[10]) → Index Error: list index out of range

(2) By using Slice Operator:

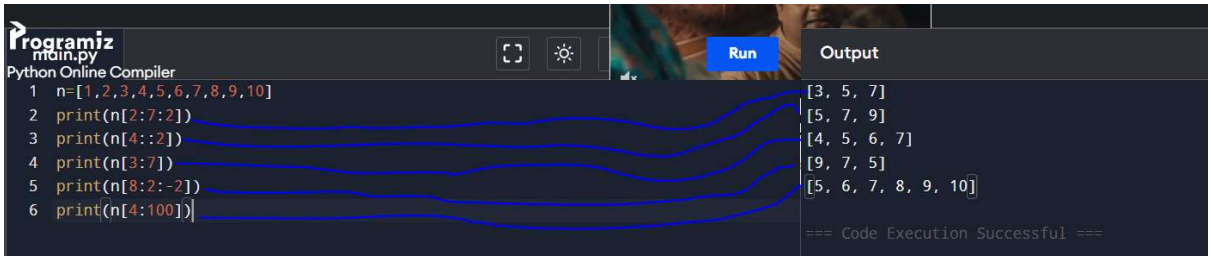
Syntax: list2 = list1[start: stop: step]

Start → It indicates the Index where slice has to Start Default Value is 0

Stop → It indicates the Index where slice has to End Default Value is max allowed Index of List i:e Length of the List

Step → increment value Default Value is 1

```
n=[1,2,3,4,5,6,7,8,9,10]
print(n[2:7:2])
print(n[4::2])
print(n[3:7])
print(n[8:2:-2])
print(n[4:100])
```



List vs Mutability:

Once we creates a List object, we can modify its content.

Hence List objects are mutable.

```
n=[10,20,30,40]
print(n)
n[1]=777
print(n)
```

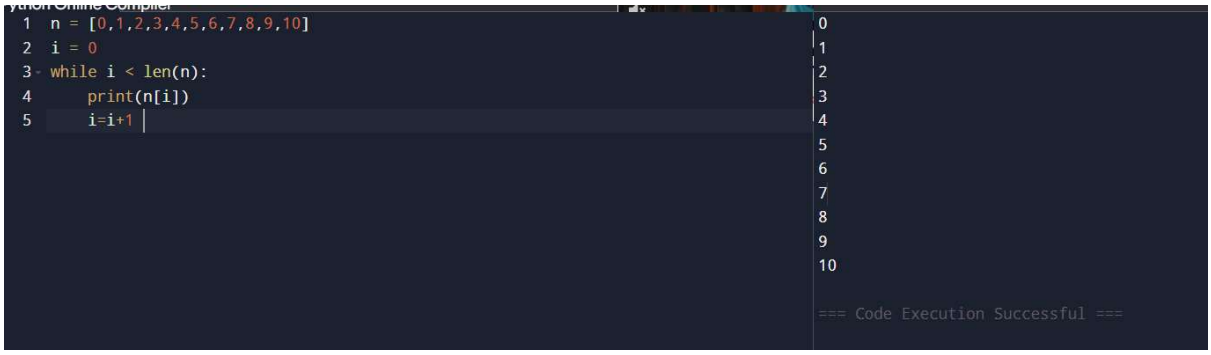
Traversing the Elements of List:

The sequential access of each element in the list is called traversal.

(1) By using while Loop:

example

```
n = [0,1,2,3,4,5,6,7,8,9,10]
i = 0
while i < len(n):
    print(n[i])
    i=i+1
```



2) By using for Loop:

example

```
n=[0,1,2,3,4,5,6,7,8,9,10]
for n1 in n:
    print(n1)
```

```
n=[0,1,2,3,4,5,6,7,8,9,10]
for n1 in n:
    print(n1)
```

```
0
1
2
3
4
5
6
7
8
9
10

=== Code Execution Successful ===
```

Example:

To display only Even Numbers:

```
n=[0,1,2,3,4,5,6,7,8,9,10]
for n1 in n:
    if n1%2==0:
        print(n1)
```

Python Online Compiler

```
n=[0,1,2,3,4,5,6,7,8,9,10]
for n1 in n:
    if n1%2==0:
        print(n1)
```

```
0
2
4
6
8
10

=== Code Execution Successful ===
```

Example

To display Elements by Index wise:

```
l = ["A","B","C"]
x = len(l)
for i in range(x):
    print(l[i],"is available at positive index: ",i,"and at negative index: ",i-x)
```

```
main.py
l = ["A","B","C"]
x = len(l)
for i in range(x):
    print(l[i],"is available at positive index: ",i,"and at negative index: ",i-x)
```

```
Output
A is available at positive index:  0 and at negative index:  -3
B is available at positive index:  1 and at negative index:  -2
C is available at positive index:  2 and at negative index:  -1

=== Code Execution Successful ===
```

Important Functions of List:

1) len():

Returns the number of elements present in the list

Eg:

n = [10, 20, 30, 40]

print(len(n) → 4

2) count():

It returns the number of occurrences of specified item in the list

n=[1,2,2,2,2,3,3]

print(n.count(1)) → 1

print(n.count(2)) → 4

print(n.count(3)) → 2

print(n.count(4)) → 0

3) index():

Returns the index of first occurrence of the specified item.

n = [1, 2, 2, 2, 2, 3, 3]

print(n.index(1)) → 0

print(n.index(2)) → 1

print(n.index(3)) → 5

print(n.index(4)) → ValueError: 4 is not in list

4) append() Function:

We can use append() function to add item at the end of the list.

```
list=[]
list.append("A")
list.append("B")
list.append("C")
print(list)
```

output => list = ['A', 'B', 'C']

Example:

To add all elements to list upto 100 which are divisible by 10

```
list=[]
for i in range(101):
   if i%10==0:
      list.append(i)
print(list)
```

5. insert() Function:

To insert item at specified index position

n=[1,2,3,4,5]

I want to change value of 1st index to "Python".
current value is 2.

```
n.insert(1,"Python")
print(n)
```

```
n=[1,2,3,4,5]
n.insert(1,"Python")
print(n)
```

```
[1, 'Python', 2, 3, 4, 5]

=== Code Execution Successful ===
```

- I want to insert “Java” at 10th index.

Given list, n = [1, ‘python’, 3, 4, 5]

We have only 5 items in the list, there is no 10th index, how to do?

n.insert(10, “Java”)
print(n)

```
n=[1,'python',3,4,5]
n.insert(10,"Java")
print(n)
```

```
[1, 'python', 3, 4, 5, 'Java']

=== Code Execution Successful ===
```

I don’t have 10th index, but I will keep you at last and provide you index value.

Print(n)
[1, 'python', 3, 4, 5, 'Java']

- I want to insert “Java” at -10th index.

N = [1, 'python', 3, 4, 5, 'Java']

n.insert(-10, “Javascript”)

I don't have minus 10th index, but I will keep you at first and shift other value which were already available to right side.

Print (n)

['Typescript', 1, 'python', 3, 4, 5, 'Java']

```
n=[1, 'python', 3, 4, 5, 'Java']
n.insert(-10,"Typescript")
print(n)
```

```
['Typescript', 1, 'python', 3, 4, 5, 'Java']

=== Code Execution Successful ===
```

Note:

1. If the specified index is greater than max index then element will be inserted at last position.
2. If the specified index is smaller than min index then element will be inserted at first position.

6. extend() Function:

To add all items of one list to another list.

l1.extend(l2)

all items present in l2 will be added to l1

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]
list1.extend(list2)
print(list1)
# Output: [1, 2, 3, 4, 5, 6]


list1 = ['h', 'i']
string1 = 'hello'
list1.extend(string1)
print(list1)
# Output: ['h', 'i', 'h', 'e', 'l', 'l', 'o']
```

```
my_list = ['a', 'b']
my_tuple = ('c', 'd')
my_list.extend(my_tuple)
print(my_list)
# Output: ['a', 'b', 'c', 'd']
```

7 . remove() Function:

1. We can use this function to remove specified item from the list.

2. If the item present multiple times then only first occurrence will be removed.

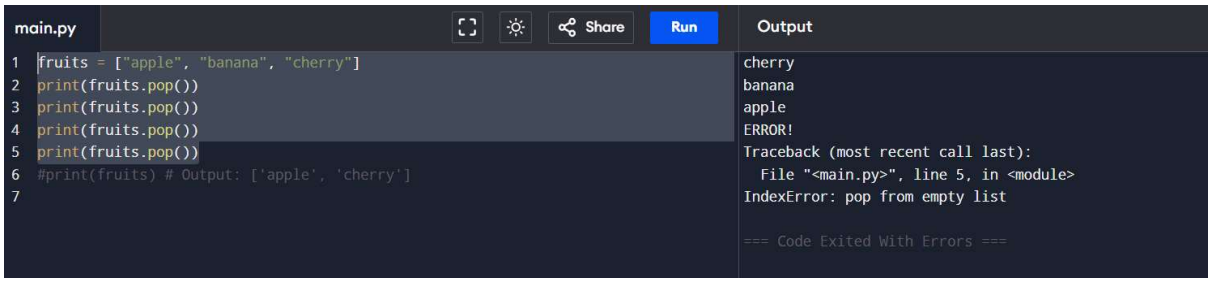
3. The syntax is `list.remove(value)`.

```
# 1. Removing a specified item
fruits = ["apple", "banana", "cherry"]
fruits.remove("banana")
print(fruits) # Output: ['apple', 'cherry']


# 2. Only the first occurrence is removed
numbers = [1, 2, 3, 2, 4]
numbers.remove(2)
print(numbers) # Output: [1, 3, 2, 4]
```

8. pop() Function:

- It removes and returns the last element of the list.
- This is only function which manipulates list and returns some element.

```
fruits = ["apple", "banana", "cherry"]
print(fruits.pop())
print(fruits.pop())
print(fruits.pop())
print(fruits.pop())
```



In general we can use pop() function to remove last element of the list. But we can use to remove elements based on index.

n.pop(index)

To remove and return element present at specified index.

```
fruits = ["apple", "banana", "cherry"]

print(fruits.pop(1))

print(fruits)
```

```python
fruits = ["apple", "banana", "cherry"]
print(fruits.pop(1))
print(fruits)

#print(fruits) # Output: ['apple', 'cherry']
```

```
banana
['apple', 'cherry']

=== Code Execution Successful ===
```

9. reverse():

We can use to reverse() order of elements of list.

fruits = ["apple", "banana", "cherry"]
fruits.reverse()
print(fruits)

n = [10, 20, 30, 40, 50]
n.reverse()
print(n)

```python
fruits = ["apple", "banana", "cherry"]
fruits.reverse()
print(fruits)



n = [10, 20, 30, 40, 50]
n.reverse()
print(n)
```

```
['cherry', 'banana', 'apple']
[50, 40, 30, 20, 10]

=== Code Execution Successful ===
```

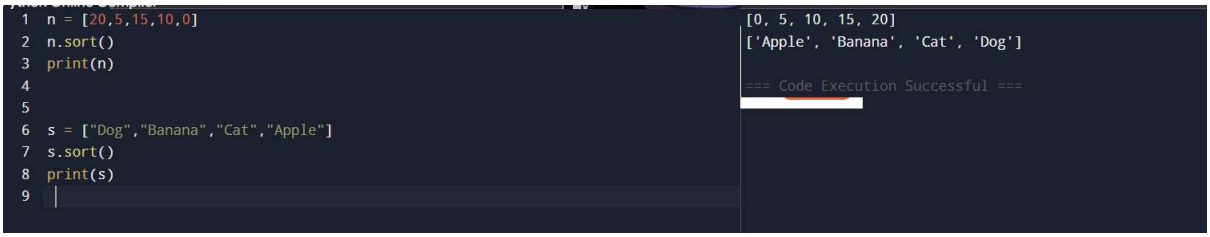
10. sort():

In list by default insertion order is preserved.

If want to sort the elements of list according to default natural sorting order then we should go for sort() method.

- For numbers → Default Natural sorting Order is Ascending Order
- For Strings → Default Natural sorting order is Alphabetical Order

```
n = [20,5,15,10,0]
n.sort()
print(n)


s = ["Dog","Banana","Cat","Apple"]
s.sort()
print(s)
```




```
n=[20,10,"A","B"]
n.sort()
print(n)
```

output => TypeError: '<' not supported between instances of 'str' and 'int'

compulsory list should contain only homogeneous elements. Otherwise we will get TypeError

To Sort in Reverse of Default Natural Sorting Order:

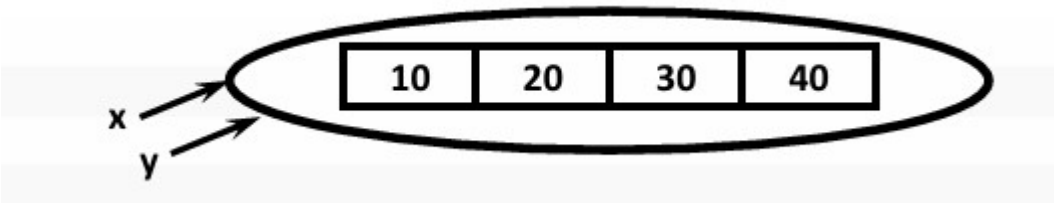
We can sort according to reverse of default natural sorting order by using reverse=True argument.

```
n = [40,10,30,20]
n.sort()
print(n)  → [10, 20, 30, 40]

n.sort(reverse = True)
print(n)  → [40, 30, 20, 10]

n.sort(reverse = False)
print(n)  → [10, 20, 30, 40]
```

Aliasing and Cloning of List Objects:

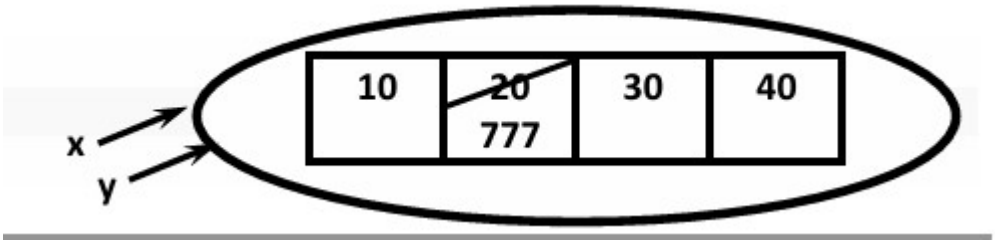
The process of giving another reference variable to the existing list is called aliasing.

```
x=[10,20,30,40]
y=x
print(id(x))
print(id(y))
```



The problem in this approach is by using one reference variable if we are changing content, then those changes will be reflected to the other reference variable.

```
x = [10,20,30,40]
y = x
y[1] = 777

print(x)
print(y)
```



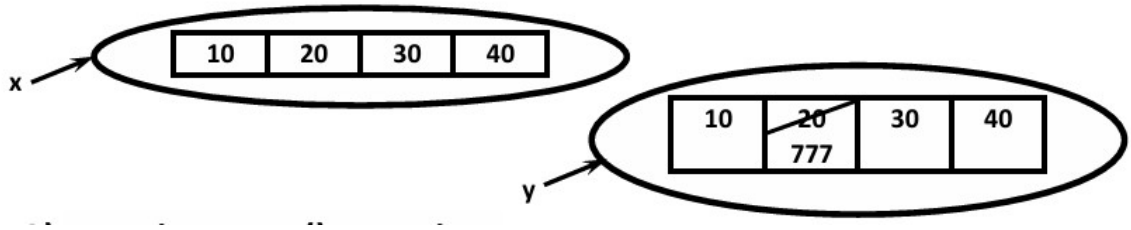
To overcome this problem we should go for cloning.

The process of creating exactly duplicate independent object is called cloning.

We can implement cloning by using slice operator or by using copy() function.

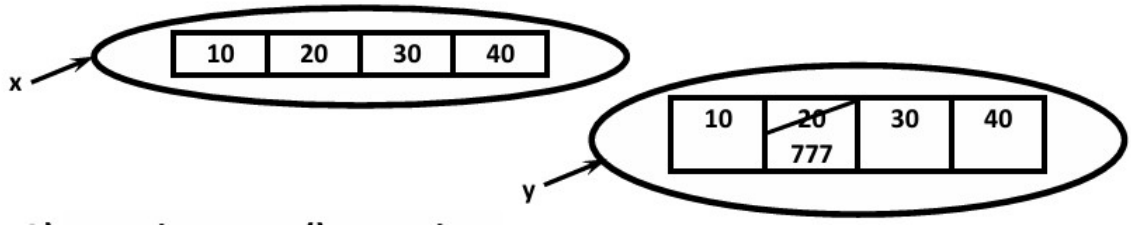
a. Using slice operator

```
x = [10,20,30,40] 2)
y = x[:]
y[1] = 777
print(x)
print(y)
```



b. By using copy() Function:

```
x = [10,20,30,40] 2)
y = x.copy()
y[1] = 777
print(x)
print(y)
```



Ques

Difference between = Operator and copy() Function

= Operator meant for aliasing
copy() Function meant for cloning

- Using Mathematical Operators for List Objects:

We can use + and * operators for List objects.

1. Concatenation (+)

```
a = [10, 20, 30]
b = [40, 50, 60]
c = a+b
print(c)  → [10, 20, 30, 40, 50, 60]
```

2. Repetition (-)

```
x = [10, 20, 30]
y = x*3
print(y) → [10, 20, 30, 10, 20, 30, 10, 20, 30]
```

Comparing List Objects

We can use comparison operators for List objects.

1. Whenever we are using
   comparison operators (==, !=)
   for List objects then the following should be considered
   1) The Number of Elements
   2) The Order of Elements
   3) The Content of Elements (Case Sensitive)

```
list1 = [10, 20, "Python"]
list2 = [10, 20, "python"]
list3 = [20, 10, "Python"]
list4 = [10, 20]
list5 = [10, 20, "Python"]

print(list1 == list2) # False (P vs p)
print(list1 == list3) # False (Order differs)
print(list1 == list4) # False (Size differs)
print(list1 == list5) # True  (Perfect match)
```

2. When ever we are using relatational Operators (<=, >, >=) between List Objects, only 1ST Element comparison will be performed.

```
x = [50, 20, 30]
y = [40, 50, 60, 100, 200]
print(x>y) → True
print(x>=y) → True
print(x<y) → False
print(x<=y) → False
```

```
x = ["Dog", "Cat", "Rat"]
y= ["Rat", "Cat", "Dog"]
print(x>y) → False
print(x>=y) → False
print(x<=y) →  True
```

Membership Operators use in List:

We can check whether element is a member of the list or not by using membership operators.

1) in Operator

2) not in Operator

```
n= [10,20,30,40]
print (10 in n) → True
print (10 not in n)  → False
print (50 in n)   → False
print (50 not in n)  → True
```

clear() Function:

We can use clear() function to remove all elements of List.

```
n=[10,20,30,40]
print(n) → [10, 20, 30, 40]

n.clear()
print(n) → []
```

Nested Lists:

Sometimes we can take one list inside another list.

Such type of lists are called nested lists.

```
n=[10,20,[30,40]]
print(n) → [10, 20, [30, 40]]
print(n[0]) → 10
print(n[2]) → [30, 40]
print(n[2][0]) → 30
print(n[2][1]) → 40
```

Ques:

Why do some Strings or Integers have the same ID But
Same list different id.

Example:

```
a = 10
b = 10

print(id(a))
print(id(b))
```

It will be same id.

```
a = []
b = []

print(id(a))
print(id(b))
```

Both id will be different.

integers and strings are immutable (cannot be changed), Python saves memory by pointing both variables to the same pre-existing object.

List is Mutable , Always get a New ID for safety.

List Comprehensions:

It is very easy and compact way of creating list objects from any iterable objects

Syntax:

list = [expression for item in list if condition]

s = [x*x for x in range(1,11)]

print(s)

output → [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

Ques:

write a program to print ip using list comprehension python.

Example ip

ip = [190.156.23.7]

numbers should be always unique after print.

```
import random
random_ips = [".".join(str(random.randint(0, 255)) for _ in range(4))]
print(random_ips)
```

Examples

Description	For Loop Equivalent	List Comprehension

Basic list creation (0 to 9)	`squares = []`
	`for x in range(10):`
	`squares.append(x**2)`
		`squares = [x**2 for x in range(10)]`

Filtering elements (even numbers)	`even_nums = []`
	`for n in numbers:`
	`if n % 2 == 0:`
	`even_nums.append(n)`
		`even_nums = [n for n in numbers if n % 2 == 0]`

Filtering and transforming (uppercase fruits with 'a')	`newlist = []`
	`for x in fruits:`
	`if "a" in x:`
	`newlist.append(x.upper())`
		`newlist = [x.upper() for x in fruits if "a" in x]`

If-else expression (labeling odd/even)	`result = []`
	`for n in a:`
	`if n % 2 == 0:`
	`result.append('Even')`
	`else:`
	`result.append('Odd')`
		`result = ['Even' if n % 2 == 0 else 'Odd' for n in a]`

Nested loops (coordinate pairs)	`combs = []`
	`for x in [1, 2, 3]:`
	`for y in [3, 1, 4]:`
	`if x != y:`
	`combs.append((x, y))`
		`combs = [(x, y) for x in [1, 2, 3] for y in [3, 1, 4] if x != y]`

Ques:

Write a Program to display Unique Vowels present in the given Word?

User has given input → "I am learning Python and Eating Apple"

text = "I am learning Python and Eating Apple"

This is a string.

```
vowels=['a','e','i','o','u']
word=input("Enter the word to search for vowels: ")
word = word.lower()
found=[]
for letter in word:
        if letter in vowels:
                if letter not in found:
                        found.append(letter)
print(found)
```

main.py

```
vowels=['a','e','i','o','u']
word=input("Enter the word to search for vowels: ")
word = word.lower()
found=[]
for letter in word:
    if letter in vowels:
        if letter not in found:
            found.append(letter)
print(found)
```

Output

```
Enter the word to search for vowels: I am learning Python and Eating Apple
['i', 'a', 'e', 'o']
['i', 'a', 'e', 'o']

=== Code Execution Successful ===
```

Ques:

Write a table of 2 using list comprehension?

Write the cube of numbers between 1 to 5 using list comprehension?

```
# Table of 2 using list comprehension
table_of_2 = [2 * i for i in range(1, 11)]

print(table_of_2)
```

 # Cubing numbers 1 through 5

cubes = [i3 for i in range(1, 6)]**

print(cubes)

TUPLE

- Tuple is exactly same as List except that it is immutable.

  i.e once we create Tuple object, we cannot perform any changes in that object.

- Hence Tuple is Read only version of List.
- If our data is fixed and never changes then we should go for Tuple.
- Insertion Order is preserved
- Duplicates are allowed
- Heterogeneous objects are allowed.
- Support indexing.
- Tuple support both +ve and -ve index. +ve index means forward direction (from left to right) and -ve index means backward direction (from right to left)
- We can represent Tuple elements within Parenthesis and with comma separator.
- Parenthesis are optional but recommended to use.

Example 1

t=10,20,30,40

print(t) → (10, 20, 30, 40)

print(type(t)) → <class 'tuple'>

Example 2

t=()

print(t) → ()

print(type(t)) → <class 'tuple'>

Example 3

t=(10)

print(t) → 10

print(type(t)) → <class 'int'>

Example 4

t=(10,)

print(t) → (10,)

print(type(t)) → <class 'int'>

Tuple Creation:

1) **t = ()**
   Creation of Empty Tuple

2) **t = (10,)**
   t = 10, Creation of Single Valued Tuple, Parenthesis are Optional, should ends with Comma

3) **t = 10, 20, 30**

   t = (10, 20, 30) Creation of multi values Tuples & Parenthesis are Optional.

4) **By using tuple ()**
   list= [10,20,30]
   t=tuple(list)
   print(t)

5) **t=tuple (range (10,20,2))**

   print(t)

Accessing Elements of Tuple:

We can access either by index or by slice operator

1) By using Index:

t = (10, 20, 30, 40, 50, 60)

print(t[0]) → 10

print(t[-1]) → 60

print(t[100]) → IndexError: tuple index out of range

2) By using Slice Operator:

t= (10,20,30,40,50,60)

print(t[2:5]) → (30, 40, 50)

print(t[2:100]) → (30, 40, 50, 60)

print(t[::2]) → (10, 30, 50)

Tuple vs Immutability:

- Once we create tuple, we cannot change its content.
- Hence tuple objects are immutable.

t = (10, 20, 30, 40)

t[1] = 70 → TypeError: 'tuple' object does not support item assignment

Mathematical Operators for Tuple:

We can apply + and * operators for tuple

1) Concatenation Operator (+):

t1= (10,20,30)

t2= (40,50,60)

t3=t1+t2

print(t3) → (10,20,30,40,50,60)

2) Multiplication Operator OR Repetition Operator (*)

t1= (10,20,30)

t2=t1*3

print(t2) → (10,20,30,10,20,30,10,20,30)

Important Functions of Tuple:

1. len()
   To return number of elements present in the tuple.

   t = (10,20,30,40)
   print(len(t)) → 4

2. count ()
   To return number of occurrences of given element in the tuple

   t = (10, 20, 10, 10, 20)
   print(t.count(10)) → 3

3. index ()
   - Returns index of first occurrence of the given element.
   - If the specified element is not available, then we will get ValueError.

   t = (10, 20, 10, 10, 20)

   print(t.index(10)) → 0

   print(t.index(30)) → ValueError: tuple.index(x): x not in tuple

4. Sorted ()
   To sort elements based on default natural sorting order

   t= (40,10,30,20)
   t1=sorted(t)
   print(t1)
   print(t)

- `t1` becomes `[10, 20, 30, 40]` (**List**)
- `t` remains `(40, 10, 30, 20)` (**Tuple**)

A **tuple** (like `t`) is immutable, meaning it cannot be changed. Since sorting requires rearranging elements, Python creates a new container. So output is list.

5. min() And max() Functions:
   These functions return min and max values according to default natural sorting order.

   t = (40,10,30,20)
   print(min(t)) → 10
   print(max(t)) → 40

Tuple Packing and Unpacking:

We can create a tuple by packing a group of variables.

a = 10
b = 20
c = 30
d = 40
t = a, b, c, d
print(t) → (10, 20, 30, 40)

- Here a, b, c, d are packed into a Tuple t. This is nothing but Tuple packing.
- Tuple unpacking is the reverse process of Tuple packing.

Note:

At the time of tuple unpacking the number of variables and number of values should be same, otherwise we will get ValueError.

t = (10,20,30,40)

a, b, c = t → ValueError: too many values to unpack (expected 3)

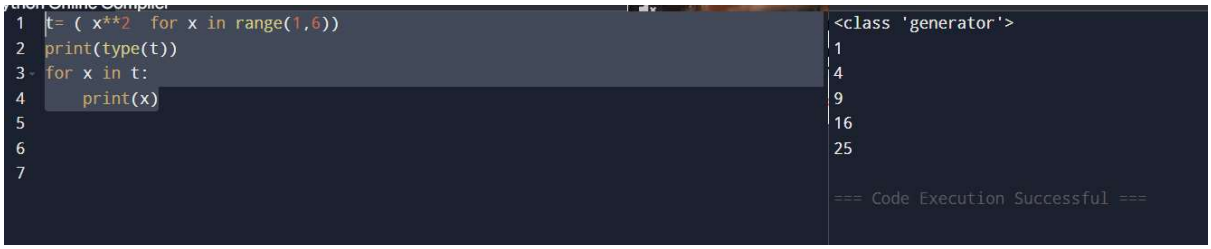
Tuple Comprehension:

Tuple Comprehension is not supported by Python.

t = (x**2 for x in range(1,6))

Here we are not getting tuple object and we are getting generator object.

```
t= ( x**2  for x in range(1,6))
print(type(t))
for x in t:
    print(x)
```



Ques

Write a Program to take a Tuple of Numbers from the Keyboard and Print its Sum and Average?

```
t=eval(input("Enter Tuple of Numbers:"))
l=len(t)
sum=0
for x in t:
    sum=sum+x
print(f"The Sum is {sum}")
print(f"The Average is {sum/l}")
```

```
t=eval(input("Enter Tuple of Numbers:"))
l=len(t)
sum=0
for x in t:
    sum=sum+x
print(f"The Sum is {sum}")
print(f"The Average is {sum/l}")
```

```
Enter Tuple of Numbers:(10,20,30,40)
The Sum is 100
The Average is 25.0

=== Code Execution Successful ===
```

Differences between List and Tuple:

Feature	List	Tuple
Mutability	**Mutable** (can add, remove, or change items).	**Immutable** (cannot be changed after creation).
Syntax	Uses square brackets `[1, 2, 3]`.	Uses parentheses `(1, 2, 3)`.
Memory	Uses **more memory**.	Uses **less memory** (more efficient).
Performance	Slower because of extra overhead.	**Faster** for iterating and accessing data.
Methods	Many methods like `.append()`, `.pop()`, `.sort()`.	Very few methods (only `.count()` and `.index()`).

Usage	Best for collections that will **change**.	Best for **fixed data** (like coordinates or DB records).

t=(10,23)

print(dir(t))

```
t=(10,23)
print(dir(t))
```

```
['__add__', '__class__', '__class_getitem__', '__contains__', '__delattr__', '__dir__'
    , '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__getitem__',
    '__getnewargs__', '__getstate__', '__gt__', '__hash__', '__init__',
    '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mul__',
    '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__rmul__',
    '__setattr__', '__sizeof__', '__str__', '__subclasshook__', 'count', 'index']
```

t=[10,23]

print(dir(t))

```
t=[10,23]
print(dir(t))
```

```
['__add__', '__class__', '__class_getitem__', '__contains__', '__delattr__',
    '__delitem__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__',
    '__getattribute__', '__getitem__', '__getstate__', '__gt__', '__hash__',
    '__iadd__', '__imul__', '__init__', '__init_subclass__', '__iter__', '__le__',
    '__len__', '__lt__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__'
    , '__repr__', '__reversed__', '__rmul__', '__setattr__', '__setitem__',
    '__sizeof__', '__str__', '__subclasshook__', 'append', 'clear', 'copy', 'count',
    'extend', 'index', 'insert', 'pop', 'remove', 'reverse', 'sort']

=== Code Execution Successful ===
```